Génie logiciel — cycle de vie et méthodes

Informatique — Licence 2 (L2)

Cours INF-L2-10

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours
Comprendre comment naît, vit et évolue un logiciel professionnel : cycle de vie, modèles de développement (cascade, cycle en V, agile), spécification des besoins, conception, tests, documentation, maintenance et initiation à Scrum.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Le cycle de vie d'un logiciel
• Leçon 2 — Modèles de développement : cascade, cycle en V et agile
• Leçon 3 — Analyse et spécification des besoins
• Leçon 4 — Conception générale et conception détaillée
• Leçon 5 — Tests logiciels : unitaires, d'intégration et fonctionnels
• Leçon 6 — Documentation et maintenance des logiciels
• Leçon 7 — Introduction aux méthodes agiles et à Scrum
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

En première année, tu as appris à écrire des programmes qui fonctionnent sur ta machine, pour un exercice, le temps d'un devoir. Un logiciel professionnel, lui, doit fonctionner pendant des mois ou des années, entre les mains d'utilisateurs qui ne sont pas informaticiens, tout en étant corrigé, amélioré et parfois repris par des développeurs qui n'ont pas écrit la première ligne. Passer du « programme qui marche » au « logiciel qui dure » : c'est exactement l'objet du génie logiciel, la discipline qui organise la production du logiciel comme on organise la construction d'un bâtiment. On n'imagine pas un maçon qui poserait les murs avant de savoir combien d'étages veut le client, ni un chantier sans plans, sans contrôle de qualité et sans réception des travaux : le génie logiciel applique au logiciel cette même rigueur, avec ses phases, ses livrables et ses vérifications.

Ce cours suit la vie complète d'un logiciel, de l'idée de départ jusqu'à son retrait, en passant par l'analyse des besoins, la conception, la réalisation, les tests, la mise en service et la maintenance. Nous appliquerons chaque notion à un même fil rouge : GestStock, une application de gestion de stock et de facturation développée pour Sahel Négoce, une petite entreprise de commerce de Bamako. Ce fil rouge te permettra de voir comment les documents produits à une phase servent directement à la phase suivante. La spécification rédigée au début, par exemple, deviendra la référence des tests, puis le support du dialogue avec la cliente lors de la recette ; le journal de maintenance, lui, racontera l'histoire du produit des années plus tard. Rien de ce qui est produit dans un projet bien mené n'est du papier perdu.

À la fin du cours, tu sauras choisir entre un modèle en cascade, un cycle en V ou une approche agile selon le contexte, rédiger des exigences testables, décrire une architecture en modules, construire un plan de test par niveau et participer à un projet mené en Scrum. Ce sont les compétences de base attendues d'un développeur junior en entreprise, bien avant la maîtrise de tel ou tel langage : les langages changent, la méthode de travail reste. Le cours alterne explications, documents de travail réels (spécifications, plans de test, backlogs) et exercices corrigés, parce que le génie logiciel ne s'apprend pas seulement en lisant : il s'apprend en produisant les livrables d'un projet, même petit, avec la rigueur qu'exigerait un vrai client.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Décrire les phases du cycle de vie d'un logiciel et le livrable produit par chacune.
- Comparer les modèles en cascade, en cycle en V et les approches agiles, puis choisir le modèle adapté à un contexte donné.
- Distinguer besoins fonctionnels et non fonctionnels, et rédiger des exigences numérotées, claires et testables.
- Différencier la conception générale (architecture) de la conception détaillée.
- Classer les tests en unitaires, d'intégration et fonctionnels, et écrire un cas de test complet.
- Identifier les types de maintenance (corrective, adaptative, évolutive, préventive) et le rôle de la documentation.
- Expliquer les rôles, les événements et les artefacts de Scrum, et rédiger une user story correcte.

Leçon 1 — Le cycle de vie d'un logiciel

Le cycle de vie d'un logiciel est l'ensemble des grandes étapes qu'il traverse, depuis l'idée initiale jusqu'à son abandon définitif. On distingue classiquement six phases : l'étude (ou analyse) des besoins, la conception, la réalisation (le codage), les tests et la validation, le déploiement (mise en production), puis l'exploitation accompagnée de la maintenance. Vient enfin, parfois après de longues années, le retrait du logiciel, remplacé par un produit plus moderne ou devenu inutile.

Chaque phase produit un livrable précis, c'est-à-dire un document ou un résultat vérifiable qui sert de point de départ à la phase suivante. L'analyse produit le cahier des charges et la spécification des besoins ; la conception produit les documents d'architecture et les maquettes ; la réalisation produit le code source ; les tests produisent des rapports de test ; le déploiement produit une version installée et un manuel utilisateur. Si un livrable est bâclé, c'est toute la suite du projet qui en paie le prix : on ne construit pas un étage solide sur des fondations approximatives.

La grande leçon du cycle de vie tient en un chiffre : une erreur coûte d'autant plus cher qu'elle est découverte tard. Une ambiguïté corrigée dans le cahier des charges demande quelques minutes de réunion ; la même erreur découverte après la mise en production peut exiger de reprendre la conception, le code et les tests, sans compter les utilisateurs mécontents. C'est pour cela que les méthodes de développement, sujet

des prochaines leçons, cherchent toutes à détecter les problèmes le plus tôt possible et à valider chaque phase avant de passer à la suivante.

L'essentiel
Le cycle de vie enchaîne : besoins → conception → réalisation → tests → déploiement → maintenance → retrait. Chaque phase produit un livrable qui engage la suivante, et une erreur détectée tard coûte beaucoup plus cher qu'une erreur détectée tôt.

Exemple — Les phases de GestStock et leurs livrables

```
PROJET GESTSTOCK — Sahel Négoce (PME, Bamako)

Phase Livrable principal
----------------- -------------------------------------
1. Étude Cahier des charges validé par la
gérante (problèmes du cahier papier)
2. Conception Architecture en 4 modules, schéma
de la base de données, maquettes
3. Réalisation Code source des modules (stocks,
facturation, clients, rapports)
4. Tests Rapport de tests + liste des
anomalies corrigées
5. Déploiement Logiciel installé sur le poste de
la boutique + manuel utilisateur
6. Maintenance Journal des corrections et des
évolutions (voir leçon 6)
7. Retrait Prévu le jour où une version web
remplacera la version actuelle
```

À retenir

- Le cycle de vie couvre le logiciel de l'idée initiale jusqu'à son retrait, pas seulement jusqu'au codage.
- Chaque phase produit un livrable vérifiable : sans livrable clair, la phase suivante démarre sur du sable.
- Règle d'or : plus une erreur est détectée tard, plus sa correction coûte cher.

Exercice

Classe chacune des activités suivantes dans la phase du cycle de vie qui lui correspond : (a) la gérante de Sahel Négoce explique que les factures doivent porter un numéro unique ; (b) un développeur écrit la fonction qui calcule le total d'une facture ; (c) on vérifie que le stock diminue bien après chaque vente ; (d) deux ans après la mise en service, on ajoute l'impression des factures en bambara et en français ; (e) on dessine le schéma de la base de données des produits.

✔ Corrigé
(a) Étude des besoins : c'est une exigence exprimée par le client, avant toute solution technique. (b) Réalisation : on écrit du code. (c) Tests : on vérifie un comportement attendu du logiciel. (d) Maintenance (de type évolutive) : le logiciel est en service depuis longtemps et on lui ajoute une fonction. (e) Conception : le schéma de la base est une décision d'architecture, prise avant le codage. Retiens la méthode : demande-toi toujours « à quel moment de la vie du logiciel se situe cette activité, et quel livrable produit-elle ? »

Leçon 2 — Modèles de développement : cascade, cycle en V et agile

Un modèle de cycle de vie décrit la manière d'enchaîner les phases. Le plus ancien, le modèle en cascade, les enchaîne strictement l'une après l'autre : on termine entièrement l'analyse avant de concevoir, on termine la conception avant de coder, et ainsi de suite, comme une cascade qui descend marche par marche. Ce modèle est simple à gérer et convient aux projets dont les besoins sont parfaitement connus et stables dès le départ ; mais si le client découvre une erreur d'analyse au moment des tests, il faut remonter tout l'escalier, ce qui coûte très cher.

Le cycle en V reprend l'idée d'un enchaînement séquentiel, mais il insiste sur une correspondance précise : à chaque phase de la branche descendante (spécification, conception) répond une phase de test sur la branche montante. La spécification des besoins sera vérifiée par les tests de validation (la recette du client), la conception générale par les tests système (fonctionnels), la conception détaillée par les tests d'intégration, et le codage par les tests unitaires. Mieux : on prépare chaque niveau de test dès la phase correspondante, au lieu de tout découvrir à la fin. Le cycle en V est très utilisé quand la fiabilité est critique.

Les approches agiles rompent avec l'enchaînement unique : le projet avance par petites itérations de quelques semaines, qui livrent chacune un morceau de logiciel utilisable. Le client voit le produit très tôt, donne son avis, et les besoins peuvent évoluer entre deux itérations sans catastrophe. En contrepartie, ces approches exigent un client disponible et une équipe disciplinée. Il n'existe pas de « meilleur » modèle dans l'absolu : on choisit selon la stabilité des besoins, la criticité du logiciel et la disponibilité du client.

L'essentiel
Cascade : phases strictement séquentielles, adapté aux besoins stables. Cycle en V : séquentiel aussi, mais chaque phase de conception a son niveau de test miroir, préparé à l'avance. Agile : livraisons fréquentes par itérations, adapté aux besoins qui évoluent.

Exemple — Tableau comparatif des trois modèles

```
Critère Cascade Cycle en V Agile
--------------- ------------- --------------- ---------------
Enchaînement Séquentiel Séquentiel Itérations
strict + tests miroirs courtes
Retour en Très coûteux Coûteux Prévu et
arrière fréquent
1re livraison À la fin À la fin Dès la 1re
au client itération
Besoins Figés au Figés au Évolutifs,
départ départ bienvenus
Rôle des tests En fin de Préparés dès À chaque
projet la conception itération
Adapté à… Besoins Logiciels Besoins mal
stables et critiques, connus ou
bien connus fiabilité vitale changeants
```

À retenir

- Le cycle en V n'est pas itératif : c'est un modèle séquentiel enrichi par la correspondance conception ↔ tests.
- Dans le cycle en V, la branche montante ne s'improvise pas : chaque niveau de test se prépare pendant la phase miroir de la descente.
- Choisir un modèle = arbitrer entre stabilité des besoins, criticité et disponibilité du client.

Exercice

Pour chacun des trois projets suivants, choisis le modèle le plus adapté (cascade, cycle en V ou agile) et justifie ton choix en deux phrases : (1) le logiciel de pilotage automatique d'un équipement médical, dont le cahier des charges est imposé par une norme de sécurité ; (2) l'application mobile d'une start-up qui cherche encore ce que ses utilisateurs veulent vraiment ; (3) la migration, à fonctionnalités identiques, d'un ancien logiciel de paie vers une nouvelle technologie.

✔ Corrigé
(1) Cycle en V : la sécurité est critique et les besoins sont figés par une norme ; chaque exigence devra être vérifiée par un niveau de test préparé dès la conception. (2) Agile : les besoins sont inconnus et vont évoluer ; des livraisons fréquentes permettront de valider les hypothèses avec de vrais utilisateurs. (3) Cascade : le périmètre est parfaitement connu (refaire exactement la même chose), l'enchaînement séquentiel suffit et simplifie la gestion. Le point commun des trois réponses : c'est le contexte, jamais la mode, qui décide du modèle.

Leçon 3 — Analyse et spécification des besoins

L'analyse des besoins consiste à comprendre ce que le client attend réellement, avant de décider comment le réaliser. On y recueille les demandes par entretiens, observation du travail actuel et étude des documents existants. Pour GestStock, l'équipe a ainsi passé une journée dans la boutique de Sahel Négoce : elle a vu la gérante remplir des cahiers de stock à la main, perdre du temps à retrouver les prix et

découvrir les ruptures de stock trop tard. Ce sont ces problèmes concrets, et non des idées abstraites, que le logiciel doit résoudre.

La spécification transforme ce recueil en un document précis, le cahier des charges ou document de spécification, qui liste les exigences du futur logiciel. On distingue les exigences fonctionnelles — ce que le logiciel doit faire : enregistrer une vente, éditer une facture, alerter quand un stock passe sous un seuil — et les exigences non fonctionnelles — comment il doit le faire : temps de réponse, sécurité des accès, facilité d'utilisation, fonctionnement sur un ordinateur modeste. Confondre les deux familles, ou oublier la seconde, est une cause classique de logiciel « conforme sur le papier » mais inutilisable en pratique.

Une bonne exigence est numérotée, unique (une seule idée par exigence), non ambiguë et testable : on doit pouvoir décider, par un test, si elle est satisfaite ou non. « Le logiciel doit être rapide » n'est pas une exigence exploitable ; « l'affichage d'une facture doit prendre moins de 2 secondes sur le poste de la boutique » en est une. La numérotation n'est pas un détail bureaucratique : elle permettra, dans les tests, de relier chaque cas de test à l'exigence qu'il vérifie, et de prouver au client que tout ce qui a été demandé a été contrôlé.

L'essentiel
Spécifier = écrire ce que le logiciel doit faire (fonctionnel) et les qualités attendues (non fonctionnel), sous forme d'exigences numérotées, sans ambiguïté et testables. Ce document est le contrat de référence pour les tests et pour la recette du client.

Exemple — Extrait de la spécification de GestStock

```
DOCUMENT DE SPÉCIFICATION — GESTSTOCK v1.0 (extrait)

Exigences fonctionnelles
EXF-01 Le logiciel enregistre chaque vente avec la
date, le produit, la quantité et le prix.
EXF-02 À chaque vente validée, le stock du produit
est diminué automatiquement de la quantité
vendue.
EXF-03 Quand le stock d'un produit passe sous le
seuil fixé par la gérante, le produit apparaît
dans la liste « À commander ».
EXF-04 Chaque facture porte un numéro unique,
jamais réutilisé, même après annulation.

Exigences non fonctionnelles
EXNF-01 L'édition d'une facture prend moins de
2 secondes sur le poste de la boutique.
EXNF-02 Seules les personnes munies d'un mot de
passe peuvent ouvrir le logiciel.
EXNF-03 Une sauvegarde automatique des données est
créée chaque jour à la fermeture.
```

À retenir

• Fonctionnel = ce que le logiciel fait ; non fonctionnel = avec quelles qualités (rapidité, sécurité, simplicité…).

• Une exigence doit être testable : si on ne peut pas vérifier si elle est tenue, elle est mal écrite.

• La numérotation des exigences crée le lien entre le besoin du client et les tests qui le valident.

Exercice

Les trois phrases suivantes, recueillies auprès d'un client, sont de mauvaises exigences. Pour chacune, explique le défaut et réécris-la en exigence testable : (a) « Le logiciel doit être convivial pour le personnel. » (b) « Le logiciel doit gérer les clients et les fournisseurs et les produits et imprimer des rapports. » (c) « Les recherches doivent être rapides. »

✔ Corrigé
(a) Défaut : « convivial » est subjectif et non testable. Réécriture possible : « Un employé formé pendant une heure doit pouvoir enregistrer une vente en moins d'une minute, sans aide. » (b) Défaut : plusieurs idées dans une seule exigence — impossible à tester ni à suivre séparément. Il faut la découper en au moins quatre exigences distinctes (gestion des clients, des fournisseurs, des produits, édition de rapports), chacune numérotée. (c) Défaut : « rapides » n'est pas mesuré. Réécriture : « L'affichage du résultat d'une recherche de produit prend moins de 2 secondes pour un catalogue de 1 000 produits. » Le réflexe : remplacer chaque adjectif vague par un critère mesurable dans des conditions précises.

Leçon 4 — Conception générale et conception détaillée

La conception répond à la question « comment va-t-on réaliser ce qui a été spécifié ? ». Elle se déroule en deux temps. La conception générale, aussi appelée architecture, décide des grands choix : découpage du logiciel en modules, organisation de la base de données, technologies retenues, manière dont les modules communiquent. Pour GestStock, l'équipe a retenu quatre modules — Produits et stocks, Ventes et facturation, Clients et fournisseurs, Rapports — adossés à une seule base de données. Ces décisions engagent tout le projet : changer d'architecture en cours de route coûte extrêmement cher.

La conception détaillée descend ensuite au niveau de chaque module : pour chaque fonction importante, on décrit les données manipulées et l'algorithme en pseudo-code, les écrans (maquettes) et les contrôles à effectuer. C'est à ce stade qu'on décide, par exemple, comment la facture calcule sa remise, dans quel ordre les lignes sont totalisées, et ce qui se passe si la quantité demandée dépasse le stock disponible. Un développeur doit pouvoir coder à partir de la conception détaillée sans avoir à inventer les règles métier.

Le lien avec les cours de modélisation est direct : les diagrammes UML (diagramme de classes, diagrammes de cas d'utilisation) et le modèle de données sont les outils

principaux de la conception. Rappel important pour le cycle en V : c'est aussi pendant la conception que l'on prépare les tests correspondants — les tests d'intégration à partir de la conception détaillée, les tests système à partir de l'architecture et de la spécification. Conception et tests se nourrissent donc mutuellement au lieu de se succéder aveuglément.

L'essentiel
Conception générale = l'architecture (modules, base de données, technologies). Conception détaillée = le fonctionnement précis de chaque module (algorithmes, écrans, contrôles), suffisant pour coder sans improviser les règles métier.

Exemple — De l'architecture à la conception détaillée d'une facture

```
CONCEPTION GÉNÉRALE — Architecture de GestStock
[ Module Produits & stocks ] [ Module Clients ]
\ /
\ /
[ Module Ventes & facturation ]
|
[ Module Rapports ]
|
Base de données unique : tables Produit,
Client, Facture, LigneFacture, MouvementStock

CONCEPTION DÉTAILLÉE — Fonction « totaliser facture »
Entrées : liste des lignes (produit, quantité)
Règles :
1. total ← 0
2. Pour chaque ligne : total ← total + prix × quantité
3. Si total ≥ 100 000 FCFA : remise de 5 %
4. TVA de 18 % ajoutée après remise
5. Si une quantité dépasse le stock disponible :
refuser la ligne et afficher un message
Sortie : montant net à payer, arrondi au franc
```

À retenir

- L'architecture se décide tôt et se change très difficilement : c'est le niveau de la conception générale.
- La conception détaillée doit lever toutes les ambiguïtés métier avant le codage (règles de calcul, cas d'erreur).
- UML et modèles de données sont les langages de la conception ; ils servent aussi à préparer les tests.

Exercice

Pour une application de gestion des inscriptions d'une école de formation, propose : (1) un découpage en quatre modules cohérents (conception générale) ; (2) pour le module d'inscription, trois règles que la conception détaillée devra trancher avant le codage.

✔ Corrigé
(1) Modules possibles : Gestion des apprenants (fiches, documents) ; Gestion des formations et des sessions (catalogue, places disponibles) ; Inscriptions et paiements (enregistrement, reçus, échéances) ; Rapports et statistiques (effectifs, taux de remplissage, attestations). Chaque module a une responsabilité unique et partage la même base de données. (2) Règles à trancher en conception détaillée : que se passe-t-il si une session est complète au moment de l'inscription (refus ? liste d'attente ?) ; un apprenant peut-il être inscrit à deux sessions qui se chevauchent ? ; l'inscription est-elle confirmée dès le premier versement ou seulement au paiement complet ? Aucune de ces réponses ne doit être inventée par le développeur au moment de coder : c'est exactement le travail de la conception détaillée, validée avec le client.

Leçon 5 — Tests logiciels : unitaires, d'intégration et fonctionnels

Tester un logiciel, c'est l'exécuter dans des conditions choisies pour révéler ses défauts, avant que les utilisateurs ne les rencontrent. On distingue plusieurs niveaux, du plus petit au plus grand. Le test unitaire vérifie une seule unité de code — une fonction, une méthode — de manière isolée : par exemple, la fonction qui calcule le total d'une facture. Le test d'intégration vérifie que plusieurs modules assemblés coopèrent correctement : la validation d'une vente doit à la fois créer la facture et diminuer le stock. Le test fonctionnel (ou test système) vérifie, du point de vue de l'utilisateur, que le logiciel complet satisfait les exigences de la spécification : enregistrer une vente de bout en bout, comme le ferait la caissière.

À ces niveaux s'ajoute la validation (ou recette) : le client lui-même vérifie, sur des scénarios réels, que le logiciel répond à ses besoins — c'est le niveau qui correspond à la spécification dans le cycle en V. Il faut aussi pratiquer les tests de non-régression : quand on corrige un défaut ou qu'on ajoute une fonction, on rejoue les anciens tests pour s'assurer de n'avoir rien cassé ailleurs. Sans eux, chaque évolution devient un risque.

Un cas de test complet précise toujours : l'exigence ou la fonction visée, les données d'entrée, les étapes à suivre et le résultat attendu. Sans résultat attendu écrit à l'avance, il n'y a pas de test, seulement une promenade dans le logiciel. Pour être gérable, la stratégie suit souvent l'image d'une pyramide : beaucoup de tests unitaires (rapides, automatisés), un nombre moyen de tests d'intégration, et peu de tests fonctionnels complets (plus lents et plus fragiles). Dans notre fil rouge, la fonction de calcul de facture — au cœur de GestStock — sera donc couverte par des tests unitaires automatiques.

L'essentiel
Unitaire : une fonction isolée. Intégration : la coopération entre modules. Fonctionnel : le logiciel complet, du point de vue de l'utilisateur, face à la spécification. Tout cas de test énonce des entrées, des étapes et un résultat attendu ; les tests de non-régression protègent les évolutions.

Exemple — Tests unitaires Python de la fonction de facturation

```
# Module facturation.py (fonction testée)
def calculer_total(prix_unitaire, quantite, remise_pct=0):
sous_total = prix_unitaire * quantite
return sous_total - sous_total * remise_pct / 100

# Fichier test_facturation.py (bibliothèque unittest)
import unittest
from facturation import calculer_total

class TestCalculerTotal(unittest.TestCase):

def test_sans_remise(self):
self.assertEqual(calculer_total(1000, 3), 3000)

def test_avec_remise(self):
# 2 articles à 2 000 F, remise 10 % -> 3 600 F
self.assertEqual(calculer_total(2000, 2, 10), 3600)

def test_quantite_zero(self):
self.assertEqual(calculer_total(1500, 0), 0)

if __name__ == "__main__":
unittest.main()
# Résultat attendu : 3 tests exécutés, tous OK
```

À retenir

- Ne confonds pas les niveaux : unitaire = le code isolé, intégration = les modules ensemble, fonctionnel = le besoin de l'utilisateur.
- Un test sans résultat attendu précis n'est pas un test.
- Automatise les tests unitaires et rejoue-les à chaque modification : c'est la non-régression.

Exercice

Classe chacun des tests suivants au bon niveau (unitaire, intégration, fonctionnel) : (a) on vérifie que la fonction « calculer la remise » renvoie 5 % pour un total de 120 000 F et 0 % pour 50 000 F ; (b) la caissière saisit une vente complète et contrôle que la facture imprimée correspond à ce qu'elle a vendu ; (c) on vérifie qu'après validation d'une vente, la facture est créée ET le stock du produit a diminué. Puis écris un quatrième test, unitaire, pour une fonction verifier_mot_de_passe(mdp) qui exige au moins 8 caractères.

✔ Corrigé
(a) Unitaire : une seule fonction, isolée du reste. (b) Fonctionnel : scénario complet, du point de vue de l'utilisatrice, face à l'exigence d'édition de facture. (c) Intégration : le test porte précisément sur la coopération entre le module Ventes et le module Stocks. Pour (d) : cas de test unitaire — entrée : « abc » (3 caractères) ; résultat attendu : la fonction renvoie Faux ; et cas complémentaire — entrée : « bamako2026 » (10 caractères) ; résultat attendu : Vrai. Remarque que tester seulement le cas qui réussit est insuffisant : un bon test unitaire couvre aussi le cas limite (exactement 8 caractères) et le cas d'échec.

Leçon 6 — Documentation et maintenance des logiciels

Un logiciel qui n'est pas documenté est un logiciel en danger : le jour où son auteur part, personne ne sait plus comment il fonctionne ni comment le modifier sans le casser. On distingue trois grandes documentations. La documentation utilisateur explique, dans la langue de l'utilisateur, comment accomplir chaque tâche (manuel, aide en ligne). La documentation technique décrit l'architecture, la base de données et les choix de conception, pour les développeurs qui reprendront le code. La documentation d'exploitation indique comment installer, sauvegarder et restaurer le logiciel au quotidien.

La maintenance est l'ensemble des modifications apportées à un logiciel après sa mise en service ; contrairement à une idée répandue, elle représente souvent la plus grande partie de la vie et du coût total d'un logiciel. On en distingue quatre types. La maintenance corrective corrige les défauts découverts en production. La maintenance adaptative ajuste le logiciel à un environnement qui change (nouveau système d'exploitation, nouvelle réglementation). La maintenance évolutive (ou perfective) ajoute de nouvelles fonctions demandées par les utilisateurs. La maintenance préventive, enfin, améliore le code sans changer son comportement visible, afin d'éviter les problèmes futurs.

Chaque demande de maintenance doit être tracée : qui l'a demandée, pourquoi, quelle modification a été faite, quels tests ont été rejoués. Ce journal de maintenance, tenu dans le temps, est aussi précieux que le code lui-même : il raconte l'histoire du produit. Pour GestStock, la demande d'impression des factures en deux langues (leçon 1) était une maintenance évolutive, tandis qu'un changement du taux de TVA décidé par l'administration fiscale relèverait d'une maintenance adaptative : même logiciel, même équipe, mais des natures de travaux très différentes à planifier et à facturer.

L'essentiel
Documentation utilisateur, technique et d'exploitation : trois regards complémentaires sur le même logiciel. Maintenance corrective (défauts), adaptative (environnement), évolutive (nouvelles fonctions), préventive (santé du code) : identifier le type, c'est savoir planifier le travail.

Exemple — Journal de maintenance de GestStock (extraits)

```
JOURNAL DE MAINTENANCE — GESTSTOCK

Date Type Demande / intervention
---------- ------------ --------------------------------
12/03 Corrective La facture n° 214 affichait un
total faux si une ligne était
annulée. Correction + test de
non-régression ajouté.
28/05 Adaptative Le taux de TVA passe de 18 % à
19 % (décision fiscale). Taux
rendu paramétrable.
17/09 Évolutive Demande de la gérante : exporter
le rapport mensuel en PDF et
l'envoyer par messagerie.
10/11 Préventive Réorganisation du module Rapports
(code devenu illisible), sans
changement visible pour les
utilisatrices.
```

À retenir

- La maintenance n'est pas un aveu d'échec : c'est la phase la plus longue de la vie du logiciel.
- Corrective = réparer ; adaptative = suivre l'environnement ; évolutive = ajouter ; préventive = consolider.
- Sans journal ni documentation, chaque intervention devient une fouille archéologique coûteuse.

Exercice

Classe chacune de ces demandes dans le bon type de maintenance : (a) depuis la dernière mise à jour du système du poste, le logiciel ne démarre plus ; (b) les utilisatrices demandent un écran de recherche par numéro de téléphone du client ; (c) un rapport affiche un total différent de la somme des lignes ; (d) l'équipe décide de renommer et de réorganiser les fonctions du module de stock, devenu confus, avant d'y ajouter de nouvelles fonctions.

✔ Corrigé
(a) Adaptative : ce n'est ni un défaut du logiciel ni une nouveauté demandée, c'est l'environnement (le système du poste) qui a changé. Si l'enquête montrait en réalité un défaut préexistant du logiciel, on reclasserait en corrective — le classement se fait sur la cause. (b) Évolutive : nouvelle fonction demandée par les utilisatrices. (c) Corrective : un défaut avéré, visible en production, à corriger. (d) Préventive : on améliore la qualité interne du code, sans rien changer pour les utilisatrices, pour rendre les évolutions futures moins risquées. Savoir nommer le type sert à estimer, à prioriser et à expliquer au client ce qu'il paie.

Leçon 7 — Introduction aux méthodes agiles et à Scrum

Les méthodes agiles sont nées d'un constat : dans beaucoup de projets, les besoins changent en cours de route, et un plan figé dès le départ conduit à livrer, en retard, un logiciel qui ne correspond plus à l'attente. Le Manifeste Agile, publié en 2001 par des praticiens du logiciel, propose de privilégier les personnes et leurs échanges, un logiciel qui fonctionne, la collaboration avec le client et l'adaptation au changement — sans nier l'importance des plans et des documents, mais en refusant de les placer au-dessus du résultat. L'agilité n'est donc pas l'absence de méthode : c'est une méthode qui organise le changement au lieu de le subir.

Scrum est le cadre de travail agile le plus répandu. L'équipe Scrum comprend trois rôles. Le Product Owner (propriétaire du produit) représente les utilisateurs et le client : il tient la liste priorisée des besoins, le Product Backlog, et décide de ce qui a le plus de valeur. Les Développeurs réalisent le travail et s'auto-organisent pour transformer les éléments choisis en un produit utilisable. Le Scrum Master aide tout le monde à comprendre et à appliquer Scrum : il anime les événements, élimine les obstacles qui bloquent l'équipe, mais ne joue ni au chef ni au secrétaire. Une équipe Scrum est petite — en général dix personnes au maximum.

Le travail avance par Sprints, des périodes de durée fixe (au maximum un mois, très souvent deux semaines) qui produisent chacun un incrément : une version utilisable du produit. Quatre événements rythment le sprint : le Sprint Planning, qui décide du travail du sprint (au maximum 8 heures pour un sprint d'un mois, environ la moitié pour deux semaines) ; le Daily Scrum, réunion quotidienne de 15 minutes où les développeurs synchronisent leur travail ; la Sprint Review, où l'on montre l'incrément aux parties prenantes et l'on recueille leurs retours (au maximum 4 heures pour un sprint d'un mois) ; et la Sprint Retrospective, où l'équipe décide comment mieux travailler au prochain sprint (au maximum 3 heures pour un sprint d'un mois). À la fin du sprint, le cycle recommence avec un backlog réajusté : c'est ce rythme qui rend Scrum prévisible malgré des besoins évolutifs.

L'essentiel
Scrum = une petite équipe (Product Owner, Scrum Master, Développeurs), un Product Backlog priorisé, et des Sprints de durée fixe rythmés par quatre événements : Planning, Daily (15 min), Review, Retrospective. Chaque sprint livre un incrément utilisable.

Exemple — Backlog produit et sprint 1 de GestStock

```
PRODUCT BACKLOG — GESTSTOCK (priorisé par la
Product Owner : la gérante de Sahel Négoce)

US-01 En tant que caissière, je veux enregistrer
une vente pour éditer la facture du client.
US-02 En tant que gérante, je veux voir la liste des
produits sous le seuil pour commander à temps.
US-03 En tant que caissière, je veux rechercher un
produit par son nom pour aller plus vite.
US-04 En tant que gérante, je veux un rapport mensuel
des ventes pour suivre l'activité.

SPRINT 1 (2 semaines) — Objectif : vendre et facturer
Sélection : US-01, US-03
Planning (4 h) : découpage en tâches — écran de
vente, calcul du total, impression, recherche.
Daily Scrum : chaque jour à 8 h 45, 15 minutes.
Review (fin de sprint, 2 h) : démonstration de la
vente et de la facture à la gérante.
Rétrospective (1 h 30) : un point à améliorer
pour le sprint 2.
```

À retenir

- Le Product Owner priorise le backlog ; il ne distribue pas les tâches techniques, c'est l'affaire des Développeurs.
- Le Scrum Master facilite et débloque ; il n'est ni le chef de l'équipe ni le supérieur du Product Owner.
- Durées repères : sprint ≤ 1 mois, Daily Scrum = 15 minutes, Planning ≤ 8 h, Review ≤ 4 h, Retrospective ≤ 3 h (pour un sprint d'un mois).

Exercice

Une équipe débute en Scrum sur un nouveau projet. Corrige les quatre erreurs suivantes : (a) le chef de projet attribue chaque matin les tâches aux développeurs pendant le Daily Scrum, qui dure une heure ; (b) la Scrum Master décide seule de l'ordre du Product Backlog ; (c) le sprint dure six semaines « parce qu'il y a beaucoup de travail » ; (d) à la fin du sprint, l'équipe présente au client des maquettes et des documents, car « le code n'est pas encore montrable ».

✔ Corrigé
(a) Double erreur : le Daily Scrum dure 15 minutes et sert aux Développeurs à se synchroniser entre eux ; personne ne distribue les tâches, l'équipe s'auto-organise. (b) L'ordre du Product Backlog relève du Product Owner, qui maximise la valeur du produit ; la Scrum Master peut l'aider sur la technique de priorisation, mais ne décide pas à sa place. (c) Un sprint dure un mois au maximum ; un besoin de six semaines se découpe en plusieurs sprints, chacun livrant un incrément. (d) La Sprint Review présente un incrément, c'est-à-dire du logiciel qui fonctionne, même partiel ; présenter des documents à la place fait perdre le retour concret des utilisateurs, qui est la raison d'être de la revue.

Travaux dirigés (TD)

TD 1 — Spécifier le module d'inscription d'une école de formation

Une école de formation professionnelle de Bamako veut remplacer ses registres papier par un logiciel de gestion des inscriptions. Le directeur explique : « Les secrétaires doivent inscrire les apprenants aux formations, suivre les paiements qui se font souvent en plusieurs fois, et savoir en un coup d'œil quelles sessions sont pleines. Le jour de la rentrée, tout le monde arrive en même temps : il ne faut pas que le logiciel nous lâche. Et il faut que la secrétaire, qui n'est pas informaticienne, s'y retrouve. » Travail demandé : (1) rédige quatre exigences fonctionnelles numérotées et testables ; (2) rédige deux exigences non fonctionnelles numérotées, en les déduisant des propos du directeur ; (3) repère, dans la phrase du directeur, l'information qui relève d'une contrainte d'environnement plutôt que d'une exigence.

✔ Corrigé du TD 1

(1) Exigences fonctionnelles possibles : EXF-01 — Le logiciel enregistre une inscription reliant un apprenant, une session et une date ; EXF-02 — Le logiciel enregistre des paiements partiels et affiche à tout moment le reste à payer d'un apprenant ; EXF-03 — Quand le nombre d'inscrits d'une session atteint le nombre de places, la session est signalée « complète » et toute nouvelle inscription y est refusée ; EXF-04 — Le logiciel affiche pour chaque session la liste des inscrits et le nombre de places restantes. Chacune est numérotée, porte une seule idée et peut être vérifiée par un test. (2) Non fonctionnelles : EXNF-01 — Le logiciel reste utilisable le jour de la rentrée avec 5 postes travaillant en même temps (déduit de « il ne faut pas que le logiciel nous lâche », reformulé de manière mesurable) ; EXNF-02 — Une secrétaire non informaticienne, après une formation d'une demi-journée, réalise une inscription en moins de 3 minutes (déduit de « il faut qu'elle s'y retrouve »). (3) La contrainte d'environnement : les paiements en plusieurs fois et l'affluence du jour de rentrée décrivent le contexte d'utilisation ; la première devient une exigence fonctionnelle (EXF-02), la seconde une contrainte de charge qui justifie EXNF-01. Retiens la méthode : écouter le client, transformer chaque attente en phrase testable, et ne jamais laisser un adjectif (« simple », « rapide », « fiable ») sans critère mesurable.

TD 2 — Construire le plan de test de la connexion utilisateur

GestStock protège son accès par un identifiant et un mot de passe (exigence EXF-10 : « L'accès au logiciel exige un identifiant et un mot de passe valides ; après 3 échecs consécutifs, le compte est bloqué 10 minutes »). Construis un plan de test pour cette fonction : (1) deux cas de test unitaires pour la fonction verifier_mot_de_passe ; (2) un cas de test d'intégration reliant la connexion à l'ouverture du logiciel ; (3) deux cas de test fonctionnels couvrant l'exigence EXF-10, dont le cas du blocage. Présente chaque cas avec : identifiant, niveau, entrées/étapes, résultat attendu.

✔ Corrigé du TD 2
Unitaires : TU-01 — entrées : identifiant « caissiere1 », mot de passe correct ; attendu : la fonction renvoie Vrai. TU-02 — entrées : même identifiant, mot de passe volontairement faux ; attendu : la fonction renvoie Faux (et, cas limite à ajouter : mot de passe vide → Faux immédiat). Intégration : TI-01 — étapes : saisir des identifiants valides sur l'écran de connexion ; attendu : l'écran principal s'ouvre ET la session enregistre le nom de l'utilisatrice, utilisé ensuite sur les factures ; ce test vérifie la coopération entre le module d'accès et le reste du logiciel, pas seulement la fonction isolée. Fonctionnels : TF-01 — scénario complet : ouvrir GestStock, se connecter avec des identifiants valides, constater que le menu de vente est accessible ; TF-02 — scénario du blocage : saisir trois fois un mot de passe faux ; attendu : le compte est refusé même avec le bon mot de passe pendant 10 minutes, conformément à EXF-10, et un message clair l'indique. Chaque niveau répond à une question différente : la fonction est-elle juste (unitaire), les modules coopèrent-ils (intégration), le besoin du client est-il satisfait (fonctionnel) ?

Travaux pratiques (TP)

TP 1 — Simuler un sprint Scrum complet sur le projet GestStock

En équipe de 4 à 5 personnes, simulez le fonctionnement d'une équipe Scrum pendant un sprint de deux semaines (compressé en une séance de travail). Étapes : 1) Désignez un Product Owner, un Scrum Master et les Développeurs. 2) Le Product Owner rédige un Product Backlog de six user stories pour GestStock, au format « En tant que..., je veux..., afin de... », et les priorise en justifiant ses deux premiers choix. 3) Tenez un Sprint Planning simulé : choisissez les stories du sprint, fixez un objectif de sprint en une phrase, et découpez la première story en tâches. 4) Rédigez le compte rendu d'un Daily Scrum (trois questions par personne, un obstacle identifié et la décision prise). 5) Préparez la Sprint Review : liste de ce qui sera démontré, à qui, et quelles questions seront posées pour recueillir les retours. 6) Terminez par une mini-rétrospective : un point qui a bien fonctionné, un point à améliorer, une action concrète pour le sprint suivant.

✔ Attendus du TP 1
Attendus du TP : le backlog contient des user stories centrées sur l'utilisateur et sa valeur (et non des tâches techniques déguisées) ; la priorisation met en tête ce qui rend le produit utilisable (par exemple enregistrer une vente avant un rapport mensuel). L'objectif de sprint tient en une phrase (« permettre de vendre et de facturer ») et les stories choisies y contribuent toutes. Le Daily Scrum simulé reste centré sur la synchronisation (fait / à faire / obstacle) et ne se transforme pas en réunion de reporting au Scrum Master. La Review prévoit une démonstration du logiciel qui fonctionne devant la « gérante », avec des questions ouvertes sur son usage réel. La rétrospective débouche sur une seule action d'amélioration, concrète et vérifiable au sprint suivant. Erreurs qui font perdre des points : un Product Owner qui impose les solutions techniques, un sprint sans objectif, des user stories sans bénéficiaire, une rétrospective qui liste des reproches sans action décidée.

Exercices corrigés supplémentaires

Exercice 1

Cycle en V : pour chaque phase de la branche descendante, donne le niveau de test qui lui correspond sur la branche montante, et explique en une phrase pourquoi cette correspondance est utile. Phases : (a) spécification des besoins ; (b) conception générale ; (c) conception détaillée ; (d) codage.

✔ Corrigé

(a) Spécification des besoins ↔ tests de validation (recette) : on vérifie que le logiciel satisfait ce qui avait été demandé. (b) Conception générale ↔ tests système (fonctionnels) : on vérifie le comportement d'ensemble de l'architecture. (c) Conception détaillée ↔ tests d'intégration : on vérifie que les modules détaillés coopèrent comme prévu. (d) Codage ↔ tests unitaires : on vérifie chaque unité produite. Utilité de la correspondance : chaque niveau de test peut être préparé dès la phase miroir (on sait quoi tester pendant qu'on conçoit), au lieu d'être improvisé à la fin du projet ; c'est tout l'intérêt du V par rapport à une cascade simple.

Exercice 2

On te donne la règle suivante pour GestStock : « Un mot de passe est valide s'il contient au moins 8 caractères, au moins une lettre et au moins un chiffre. » Écris quatre cas de test unitaires pour la fonction mot_de_passe_valide(mdp) : un cas nominal, un cas limite, et deux cas d'échec différents. Pour chaque cas, précise l'entrée et le résultat attendu.

✔ Corrigé

Cas nominal : entrée « bamako2026 » (lettres + chiffres, 10 caractères) → Vrai. Cas limite : entrée « abcdefg7 » (exactement 8 caractères, lettres et un chiffre) → Vrai ; c'est le cas le plus précieux, car les erreurs se cachent presque toujours sur les bornes (« au moins 8 » accepté, 7 refusé). Cas d'échec n° 1 : entrée « abc123 » (6 caractères seulement) → Faux, échec sur la longueur. Cas d'échec n° 2 : entrée « uniquementlettres » (long, mais aucun chiffre) → Faux, échec sur la composition. On peut ajouter « 12345678 » (aucune lettre) → Faux. Méthode : une règle à trois conditions se teste en faisant échouer une condition à la fois, toutes les autres étant satisfaites ; ainsi, si le test échoue, on sait exactement quelle condition est mal programmée.

Exercice 3

Pour chaque situation, choisis le modèle de développement le plus adapté (cascade, cycle en V, agile) et justifie en une ou deux phrases : (a) le développement d'un jeu mobile de divertissement dont le concept sera ajusté selon les réactions des premiers joueurs ; (b) le logiciel de gestion des dossiers d'un concours national, dont le règlement fixe toutes les règles à l'avance, pour une utilisation unique l'an prochain ; (c) le remplacement d'un logiciel de laboratoire d'analyses médicales, où une erreur de résultat peut avoir des conséquences graves sur les patients.

✔ Corrigé
(a) Agile : le concept évoluera avec les retours des joueurs ; il faut livrer vite une première version jouable et itérer. (b) Cascade : les règles sont fixées par un règlement, le périmètre ne bougera pas et la date d'usage est unique ; un enchaînement séquentiel simple suffit — un cycle en V serait également défendable si l'on exige une traçabilité complète des tests, mais la criticité reste modérée. (c) Cycle en V : la criticité est maximale ; chaque exigence et chaque choix de conception doivent être associés, dès le départ, au niveau de test qui les vérifiera. La question à se poser reste la même : les besoins sont-ils stables ? Le logiciel est-il critique ? Le client peut-il participer régulièrement ?

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Quelle est la première phase du cycle de vie d'un logiciel ?

   A) Le codage

   B) L'étude et l'analyse des besoins

   C) Les tests

2. Dans le cycle en V, les tests unitaires correspondent à quelle phase de la branche descendante ?

   A) La spécification des besoins

   B) La conception générale

   C) Le codage

3. Laquelle de ces phrases est une exigence fonctionnelle ?

   A) Le logiciel doit répondre en moins de 2 secondes

   B) Le logiciel doit être facile à utiliser

   C) Le logiciel imprime une facture pour chaque vente

4. Pourquoi numérote-t-on les exigences dans une spécification ?

   A) Pour allonger le document

   B) Pour relier chaque exigence aux tests qui la vérifient

   C) Parce que le client l'exige toujours

5. Quel niveau de test vérifie la coopération entre deux modules ?

   A) Le test unitaire

   B) Le test d'intégration

   C) La recette client

6. Ajouter une fonction demandée par les utilisateurs, deux ans après la mise en service, relève de la maintenance…

   A) Corrective

   B) Adaptative

   C) Évolutive

7. Dans Scrum, qui décide de l'ordre des éléments du Product Backlog ?

A) Le Scrum Master

B) Le Product Owner

C) Les Développeurs, au vote

8. Combien de temps dure un Daily Scrum ?

A) 15 minutes

B) 1 heure

C) Le temps qu'il faut, sans limite

9. Quelle affirmation sur le modèle en cascade est exacte ?

A) Les phases se répètent par itérations d'une semaine

B) Chaque phase doit être terminée avant de passer à la suivante

C) Il n'y a pas de phase de test

10. Une user story correcte a la forme…

A) « Développer la base de données des ventes »

B) « Le système doit être rapide et sûr »

C) « En tant que caissière, je veux retrouver un produit par son nom afin de servir le client plus vite »

Corrigés du QCM

1. Réponse B — Tout commence par la compréhension du besoin ; coder avant de savoir ce qu'attend le client est la première cause d'échec des projets.

2. Réponse C — Chaque unité de code produite pendant le codage est vérifiée par les tests unitaires : c'est le bas du V.

3. Réponse C — Imprimer une facture décrit une fonction du logiciel ; les deux autres phrases sont des exigences non fonctionnelles (performance, utilisabilité).

4. Réponse B — La numérotation crée la traçabilité : chaque cas de test peut citer l'exigence qu'il contrôle, et rien n'est oublié.

5. Réponse B — Le test unitaire isole une fonction ; le test d'intégration assemble les modules ; la recette est la validation par le client du produit complet.

6. Réponse C — Corrective = corriger un défaut ; adaptative = suivre un changement d'environnement ; évolutive = ajouter des fonctions.

7. Réponse B — Le Product Owner maximise la valeur du produit et ordonne le Product Backlog ; les Développeurs choisissent comment réaliser le travail du sprint.

8. Réponse A — Le Daily Scrum est un événement de 15 minutes, à heure et lieu fixes, pour que les Développeurs synchronisent leur travail.

9. Réponse B — La cascade est un enchaînement séquentiel strict ; elle suppose des besoins stables et bien connus dès le départ.

10. Réponse C — Une user story nomme un bénéficiaire, un besoin et un bénéfice ; les deux autres propositions sont une tâche technique et un slogan non testable.

Résumé — les notions importantes

• Le cycle de vie d'un logiciel enchaîne besoins, conception, réalisation, tests, déploiement, maintenance et retrait ; chaque phase produit un livrable, et une erreur coûte d'autant plus cher qu'elle est détectée tard.

• Cascade, cycle en V et agile sont trois façons d'organiser ces phases : séquentiel strict, séquentiel avec tests miroirs préparés dès la conception, ou itérations courtes livrant un produit utilisable.

• Dans le cycle en V, chaque phase de conception a son niveau de test correspondant : codage ↔ unitaire, conception détaillée ↔ intégration, conception générale ↔ fonctionnel, spécification ↔ validation.

• Une bonne spécification contient des exigences numérotées, uniques et testables, réparties entre le fonctionnel (ce que fait le logiciel) et le non fonctionnel (ses qualités mesurables).

• La conception générale fixe l'architecture (modules, données, technologies) ; la conception détaillée précise chaque fonction au point de pouvoir coder sans improviser les règles métier.

• Tester, c'est exécuter le logiciel avec des entrées choisies et un résultat attendu ; les tests de non-régression, rejoués à chaque évolution, empêchent les corrections de casser l'existant.

• La maintenance se décline en corrective, adaptative, évolutive et préventive ; Scrum organise le travail en sprints avec trois rôles (Product Owner, Scrum Master, Développeurs), un backlog priorisé et quatre événements aux durées cadrées.

Fiches de révision express

Chaque fiche reprend les points « À retenir » d'une leçon. Relis-les la veille d'un contrôle : cinq minutes par fiche suffisent.

Fiche 1 — Le cycle de vie d'un logiciel
Le cycle de vie couvre le logiciel de l'idée initiale jusqu'à son retrait, pas seulement jusqu'au codage. ; Chaque phase produit un livrable vérifiable : sans livrable clair, la phase suivante démarre sur du sable. ; Règle d'or : plus une erreur est détectée tard, plus sa correction coûte cher.

Fiche 2 — Modèles de développement : cascade, cycle en V et agile
Le cycle en V n'est pas itératif : c'est un modèle séquentiel enrichi par la correspondance conception ↔ tests. ; Dans le cycle en V, la branche montante ne s'improvise pas : chaque niveau de test se prépare pendant la phase miroir de la descente. ; Choisir un modèle = arbitrer entre stabilité des besoins, criticité et disponibilité du client.

Fiche 3 — Analyse et spécification des besoins
Fonctionnel = ce que le logiciel fait ; non fonctionnel = avec quelles qualités (rapidité, sécurité, simplicité…). ; Une exigence doit être testable : si on ne peut pas vérifier si elle est tenue, elle est mal écrite. ; La numérotation des exigences crée le lien entre le besoin du client et les tests qui le valident.

Fiche 4 — Conception générale et conception détaillée
L'architecture se décide tôt et se change très difficilement : c'est le niveau de la conception générale. ; La conception détaillée doit lever toutes les ambiguïtés métier avant le codage (règles de calcul, cas d'erreur). ; UML et modèles de données sont les langages de la conception ; ils servent aussi à préparer les tests.

Fiche 5 — Tests logiciels : unitaires, d'intégration et fonctionnels
Ne confonds pas les niveaux : unitaire = le code isolé, intégration = les modules ensemble, fonctionnel = le besoin de l'utilisateur. ; Un test sans résultat attendu précis n'est pas un test. ; Automatise les tests unitaires et rejoue-les à chaque modification : c'est la non-régression.

Fiche 6 — Documentation et maintenance des logiciels
La maintenance n'est pas un aveu d'échec : c'est la phase la plus longue de la vie du logiciel. ; Corrective = réparer ; adaptative = suivre l'environnement ; évolutive = ajouter ; préventive = consolider. ; Sans journal ni documentation, chaque intervention devient une fouille archéologique coûteuse.

Fiche 7 — Introduction aux méthodes agiles et à Scrum
Le Product Owner priorise le backlog ; il ne distribue pas les tâches techniques, c'est l'affaire des Développeurs. ; Le Scrum Master facilite et débloque ; il n'est ni le chef de l'équipe ni le supérieur du Product Owner. ; Durées repères : sprint ≤ 1 mois, Daily Scrum = 15 minutes, Planning ≤ 8 h, Review ≤ 4 h, Retrospective ≤ 3 h (pour un sprint d'un mois).

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 2 — LES MÉTHODOLOGIES DE GESTION DE PROJET À CONNAÎTRE

Chaîne : Le chef de projet · Langue : Français · Niveau : Débutant

Un panorama clair des grandes méthodologies de gestion de projet : méthodes agiles, cascade (Waterfall), cycle en V et approches hybrides. Idéal pour situer les modèles de la leçon 2 les uns par rapport aux autres avant de les comparer en détail.

https://www.youtube.com/watch?v=5MQcmJvJUlk

Leçon 5 — 63 - Les tests version longue

Chaîne : Frédéric BAURAND (Frédéric BAURAND) · Langue : Français · Niveau : Débutant à intermédiaire

Un cours complet sur les différents types de tests qu'un projet doit intégrer : unitaires, d'intégration et fonctionnels. À regarder pour consolider la leçon 5 et savoir expliquer, exemples à l'appui, à quoi sert chaque niveau de test.

https://www.youtube.com/watch?v=rcPXs3TkrxE

Leçon 5 — TEST LOGICIEL #4/? - Mise en place des tests unitaires

Chaîne : Faisons Le Point · Langue : Français · Niveau : Intermédiaire

La mise en place concrète de tests unitaires dans un projet, avec des outils d'automatisation modernes. Le prolongement pratique de l'exemple unittest de la leçon 5 : de la théorie du cas de test à son exécution automatique.

https://www.youtube.com/watch?v=jAYekAz5E9M

Leçon 7 — Les bases de scrum : Masterclass Complète pour Débutants

Chaîne : Agitips · Langue : Français · Niveau : Débutant

Scrum expliqué de A à Z en français : pourquoi les plans traditionnels échouent, les trois rôles, les artefacts (backlogs, incrément) et le déroulement complet d'un sprint. La vidéo à voir pour ancrer toute la leçon 7.

https://www.youtube.com/watch?v=GOMf0DRcIK0

Leçon 7 — SCRUM La méthode agile en 10 minutes (Projet agile)

Chaîne : StrategeMarketing · Langue : Français · Niveau : Débutant

Dix minutes pour comprendre l'essentiel de Scrum : sprint, Product Owner, Scrum Master, user stories et burndown chart. Parfait en révision express avant un contrôle sur la leçon 7.

https://www.youtube.com/watch?v=anZcElQlpoY

Sources et références

- Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.
- Schwaber, K. & Sutherland, J. — « Le Guide Scrum » (version française, scrumguides.org) : la définition officielle et gratuite de Scrum, rôles, événements et artefacts.
- Le Manifeste pour le développement Agile de logiciels (agilemanifesto.org/fr) : les quatre valeurs et les douze principes, à lire dans le texte.
- Pressman, R. & Maxim, B. — « Software Engineering: A Practitioner's Approach » (McGraw-Hill) : l'ouvrage de référence sur les modèles de cycle de vie et les processus logiciels.
- Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés à la publication).

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !

Mes notes

Utilise ces pages pour recopier les définitions, refaire les calculs et noter tes questions pour le prochain cours.